

Συστήματα Ευφύων Πρακτόρων - 2η Ομαδική Εργασία



Μάθημα: Συστήματα Ευφύων Πρακτόρων
Διδάσκων: Γεώργιος Βούρος

Αρ.	Ονοματεπώνυμο	A.M.	E-mail
1	Δαμιανός Ζούμπος	E22056	d.zoumpos04@gmail.com
2	Γεώργιος Αζαμ	E22003	george.azam11@gmail.com
3	Άννα-Μαρία Νάιντενοβα	E22117	annamaria.ndn@gmail.com

Ημερομηνία Παράδοσης:
Φεβρουάριος 2026

Περιεχόμενα

1	Εισαγωγή	2
2	Σχεδιασμός Πολυπρακτορικού Συστήματος	2
2.1	Αντικείμενα, Εμπόδια και Στόχοι Περιβάλλοντος	2
2.2	Αναπαράσταση Περιβάλλοντος	3
2.3	Αποφυγή Συγκρούσεων στην Κίνηση	3
2.4	Πρωτόκολλο Διαπραγμάτευσης (Agent Communication Language)	3
2.4.1	Μηχανισμός Λήψης Αποφάσεων	3
2.4.2	Μηνύματα ACL και Ρόλος τους στο Πρωτόκολλο	4
2.4.3	Υπολογισμός Χρησιμότητας και Tie-breaking	4
2.4.4	Ανάλυση Πρωτοκόλλου Επικοινωνίας	5
3	Υλοποίηση σε JASON / Java	5
3.1	Μηνύματα και Συγχρονισμός	5
3.1.1	Καταστάσεις Αναμονής (Wait States) και Ασύγχρονη Επικοινωνία	6
3.1.2	Tie-breaking και Εγγύηση Προόδου	6
4	Πειραματική Μελέτη	6
4.1	Σύγκριση Μονοπρακτορικού και Πολυπρακτορικού Συστήματος	7
5	Προβλήματα κατά την Υλοποίηση και Αντιμετώπιση	7
5.1	Επικάλυψη Αντικειμένων και Στατικών Στόχων	7
5.2	Δυναμική Μετακίνηση Αντικειμένων (Stuck Agents)	8
5.3	Περιορισμοί στην Προσέγγιση Στόχων	8
6	Ανάλυση Επιτυχούς Ολοκλήρωσης	8
6.1	Διαχείριση Αποτυχιών και Μηχανισμοί Ανάνηψης (Failure Handling) . .	9
7	Συμπεράσματα	10

1 Εισαγωγή

Στη δεύτερη αυτή εργασία, επεκτείνουμε το σύστημα πρακτικού συλλογισμού (BDI) σε ένα πολυπρακτορικό περιβάλλον. Προσθέτουμε έναν δεύτερο πράκτορα (Πράκτορας 2), ο οποίος είναι κλώνος του πρώτου και ξεκινά πάντα από τη θέση (3,3). Η κύρια πρόκληση είναι ο συντονισμός των δύο πρακτόρων για την αποφυγή συγκρούσεων τόσο στην κίνηση όσο και στην ανάθεση εργασιών.

2 Σχεδιασμός Πολυπρακτορικού Συστήματος

2.1 Αντικείμενα, Εμπόδια και Στόχοι Περιβάλλοντος

Το περιβάλλον αποτελείται από πλέγμα 5×5 και περιλαμβάνει κινητά αντικείμενα, στατικά εμπόδια και δύο πράκτορες.

Αντικείμενα:

- Brush (b)
- Color (cl)
- Key (k)
- Code (cd)
- Chair (ch)
- Table (t)
- Door (d)

Τα αντικείμενα Brush και Color απαιτούνται για τη βαφή της καρέκλας και του τραπεζιού, ενώ τα Key και Code απαιτούνται για το άνοιγμα της πόρτας.

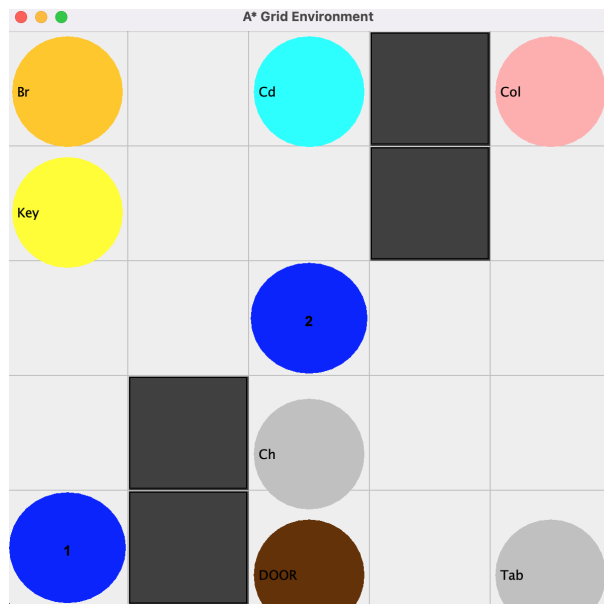
Εμπόδια: Ορισμένα κελιά του πλέγματος είναι μη προσπελάσιμα και λειτουργούν ως στατικά εμπόδια. Επιπλέον, κάθε πράκτορας αντιμετωπίζει δυναμικά τον άλλον ως προσωρινό εμπόδιο ώστε να αποφεύγονται συγκρούσεις.

Στόχοι Συστήματος: Οι βασικοί στόχοι είναι:

- Άνοιγμα της πόρτας (open_door)
- Βάψιμο τραπεζιού (paint_table)
- Βάψιμο καρέκλας (paint_chair)

Οι στόχοι κατανέμονται δυναμικά μεταξύ των πρακτόρων μέσω του πρωτοκόλλου διαπραγμάτευσης.

2.2 Αναπαράσταση Περιβάλλοντος



Σχήμα 1: Περιβάλλον πλέγματος με δύο πράκτορες, αντικείμενα, πόρτα και εμπόδια

Οι πράκτορες ξεκινούν από προκαθορισμένες θέσεις και κινούνται στο πλέγμα συλλέγοντας αντικείμενα, αποφεύγοντας εμπόδια και συντονίζοντας τις ενέργειές τους μέσω διαπραγμάτευσης.

2.3 Αποφυγή Συγκρούσεων στην Κίνηση

Οι δύο πράκτορες δεν επιτρέπεται να βρίσκονται στο ίδιο κελί. Στην υλοποίησή μας, κάθε πράκτορας θεωρεί τη θέση του άλλου ως προσωρινό εμπόδιο.

- Χρησιμοποιείται δυναμική ενημέρωση των πεποιθήσεων (beliefs) για τη θέση του άλλου πράκτορα.
- Σε περίπτωση επικείμενης σύγκρουσης, ο πράκτορας επανεκκινεί τον σχεδιασμό μονοπατιού θεωρώντας το κελί του άλλου πράκτορα ως μη προσπελάσιμο.

2.4 Πρωτόκολλο Διαπραγμάτευσης (Agent Communication Language)

Η επικοινωνία μεταξύ των δύο πρακτόρων υλοποιείται μέσω μηνυμάτων ACL και ακολουθεί ένα αυστηρό πρωτόκολλο για την κατανομή των εργασιών (open_door, paint_table, paint_chair).

2.4.1 Μηχανισμός Λήψης Αποφάσεων

Το πρωτόκολλο εξελίσσεται σε τρία βασικά στάδια:

- **Ενημέρωση (Inform):** Κάθε πράκτορας υπολογίζει τη χρησιμότητά του (U) για μια εργασία και τη γνωστοποιεί στον άλλον μέσω του μηνύματος `inform(Task, U)`. Αν ένας πράκτορας λάβει πληροφορία πριν υπολογίσει τη δική του χρησιμότητα, την υπολογίζει άμεσα και απαντά.
- **Πρόταση (Propose):** Ο πράκτορας που διαπιστώνει ότι έχει τη μεγαλύτερη χρησιμότητα στέλνει μήνυμα `propose(Task)`. Ο `main_agent` προτείνει ακόμα και σε ισοπαλία λόγω προτεραιότητας (`priority`), ενώ ο `second_agent` μόνο αν υπερτερεί αυστηρά.
- **Αποδοχή/Απόρριψη (Accept/Reject):** Ο αποδέκτης της πρότασης συγκρίνει τις τιμές. Αν συμφωνεί, στέλνει `accept` και η εργασία ανατίθεται στον άλλον (`owner(Task, other)`). Σε αντίθετη περίπτωση, στέλνει `reject` και το πρωτόκολλο επανεκκινείται.

2.4.2 Μηνύματα ACL και Ρόλος τους στο Πρωτόκολλο

Μήνυμα	Σκοπός / Επίδραση σε beliefs
<code>inform(Task, U)</code>	Ανταλλαγή τιμής χρησιμότητας. Ο παραλήπτης ενημερώνει <code>otheru(Task, Uo)</code> . Αν δεν έχει υπολογίσει ακόμα <code>myu(Task, U)</code> , την υπολογίζει και απαντά.
<code>propose(Task)</code>	Δήλωση πρόθεσης ανάληψης (<code>ownership</code>) από τον προτείνοντα. Ο παραλήπτης συγκρίνει <code>myu</code> και <code>otheru</code> και απαντά.
<code>accept(Task)</code>	Ο αποδέκτης αποδέχεται την πρόταση: ενημερώνεται <code>decided(Task, me/other)</code> και τίθεται <code>owner(Task, ...)</code> .
<code>reject(Task)</code>	Απόρριψη πρότασης: τίθεται <code>decided(Task, restart)</code> και το πρωτόκολλο επανεκκινείται με αύξηση του μετρητή N .

Πίνακας 1: ACL messages που υλοποιούν το negotiation protocol και τα βασικά beliefs που επηρεάζουν.

2.4.3 Υπολογισμός Χρησιμότητας και Tie-breaking

Η χρησιμότητα U υπολογίζεται με βάση την απόσταση Manhattan από την τρέχουσα θέση του πράκτορα (X, Y) έως το αντικείμενο στόχο (TX, TY) :

$$U = 100 - (|X - TX| + |Y - TY|)$$

Για την αποφυγή αδιεξόδων σε περιπτώσεις ισοπαλίας:

- Ο `main_agent` διαθέτει την πεποίθηση `priority` για την επίλυση συγκρούσεων.
- Η λογική `should_propose` επιτρέπει στον `main_agent` να διεκδικήσει την εργασία αν $U \geq U_o$, ενώ ο `second_agent` υποχωρεί αν $U_o \geq U$.
- Αν η διαπραγμάτευση υπερβεί τους 15 κύκλους ($N \geq 15$), ο `main_agent` αναλαμβάνει αναγκαστικά την εργασία για τη διασφάλιση της προόδου.

2.4.4 Ανάλυση Πρωτοκόλλου Επικοινωνίας

Το πρωτόκολλο διαπραγμάτευσης βασίζεται σε ανταλλαγή μηνυμάτων τύπου ACL

Η διαδικασία ξεκινά όταν ένας πράκτορας ενεργοποιήσει τον στόχο `negotiate_then_maybe_do(Task, U)`. Αρχικά, κάθε πράκτορας υπολογίζει τη χρησιμότητα του για την εργασία που έχει και αποστέλλει μήνυμα. `inform(Task, U)` στον άλλον.

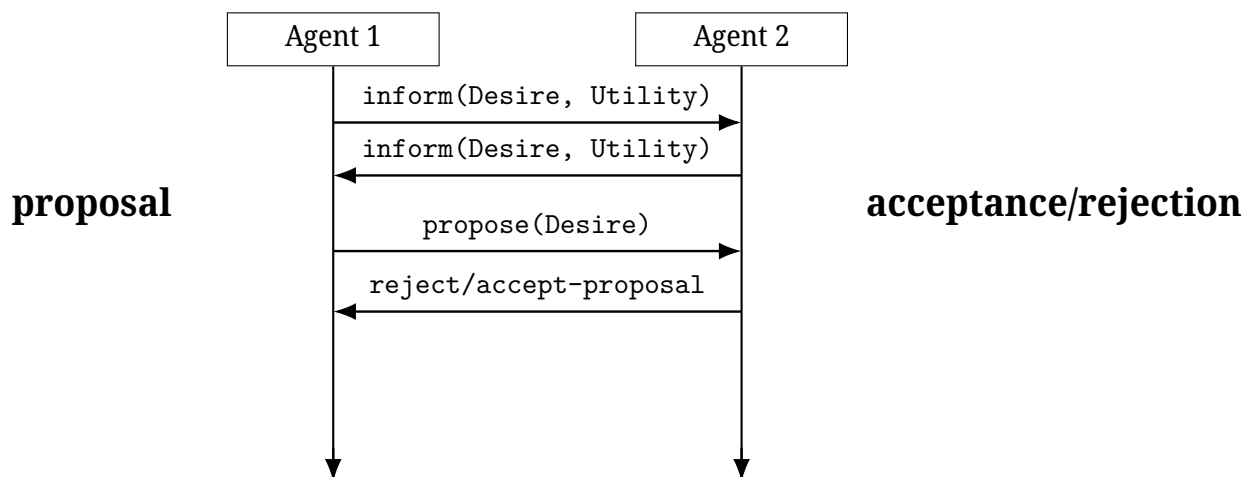
Μετά τη λήψη των δύο τιμών χρησιμότητας:

- Αν ένας πράκτορας έχει αυστηρά μεγαλύτερη χρησιμότητα, αποστέλλει `propose(Task)`.
- Ο παραλήπτης πράκτορας απαντά με `accept(Task)` or `reject(Task)` ανάλογα με τη σύγκριση των τιμών.
- Σε περίπτωση απόρριψης, το πρωτόκολλο επανεκκινείται με αυξημένο μετρητή επανλήψεων.

Για την αποφυγή αδιεξόδων, έχει οριστεί ανώτατο όριο 15 κύκλων διαπραγμάτευσης. Αν ξεπεραστεί, η εργασία ανατίθεται υποχρεωτικά στον αντίπαλο πράκτορα, εξασφαλίζοντας πρόοδο του συστήματος.

Η επικοινωνία υλοποιείται πλήρως ασύγχρονα μέσω καταστάσεων αναμονής (`wait states`), επιτρέποντας στους πράκτορες να συνεχίζουν τη λειτουργία τους χωρίς `blocking`.

Simple negotiation protocol



Σχήμα 2: Simple negotiation protocol implemented via ACL messages (`inform`, `propose`, `accept/reject`).

3 Υλοποίηση σε JASON / Java

3.1 Μηνύματα και Συγχρονισμός

Στον κώδικα `AgentSpeak`, ο συντονισμός επιτυγχάνεται μέσω των εσωτερικών ενεργειών `.send` και τη χρήση καταστάσεων αναμονής (`wait states`).

```
// A
.send(OA, tell, inform(Task, U));

// E
+propose(Task)[source(Other)] : myu(Task,U) & otheru(Task,Uo) & (Uo > U) <-
    .send(Other, tell, accept(Task));
    -+decided(Task,other); -+owner(Task,other).

// K
+!wait_other_inform(Task) : otheru(Task,_) <- true.
+!wait_other_inform(Task) <- .wait(100); !wait_other_inform(Task).
```

3.1.1 Καταστάσεις Αναμονής (Wait States) και Ασύγχρονη Επικοινωνία

Η επικοινωνία είναι ασύγχρονη και υλοποιείται μέσω wait states. Κάθε πράκτορας περιμένει γεγονότα (belief updates) που δημιουργούνται από εισερχόμενα μηνύματα:

- wait_other_inform(Task): αναμονή μέχρι να εμφανιστεί otheru(Task, _).
- wait_propose(Task, N): αναμονή μέχρι να ληφθεί propose(Task) ή να τεθεί decided(Task, ...).
- wait_accept_or_reject(Task, N): αναμονή απάντησης στην πρόταση και επανεκκίνηση αν απαιτηθεί.

Η χρήση .wait(100) αποφεύγει busy-waiting και επιτρέπει ομαλή εκτέλεση intentions.

3.1.2 Tie-breaking και Εγγύηση Προόδου

Ο main_agent έχει προτεραιότητα σε ισοπαλία (priority) ώστε να αποφεύγονται αδιέξοδα:

main: propose if $U > U_o$ ή $(U = U_o \wedge \text{priority})$

Ο second_agent προτείνει μόνο σε αυστηρή υπεροχή:

agent2: propose if $U > U_o$

Επιπλέον, υπάρχει όριο επανάληψης διαπραγμάτευσης ($N \geq 15$) για εγγύηση προόδου. Ο main_agent αναλαμβάνει αναγκαστικά την εργασία, ενώ ο second_agent την εκχωρεί στον main_agent.

4 Πειραματική Μελέτη

Το σύστημα εκτελέστηκε για 100 επεισόδια. Οι θέσεις των αντικειμένων στόχων (D, Ch, T) άλλαζαν δυναμικά σε κάθε επεισόδιο, ενώ οι αρχικές θέσεις των πρακτόρων παρέμεναν σταθερές ((1,1) και (3,3)).

Μετρική (Μέσος Όρος)	Μονοπρακτορικό	Πολυπρακτορικό
Συνολική Χρησιμότητα	...	...
Αριθμός Ενεργειών	...	...

Πίνακας 2: Σύγκριση απόδοσης

4.1 Σύγκριση Μονοπρακτορικού και Πολυπρακτορικού Συστήματος

Παρακάτω συνοψίζεται η διαφορά μεταξύ μονοπρακτορικής και πολυπρακτορικής εκτέλεσης στο ίδιο grid και με τα ίδια αντικείμενα/στόχους:

- **Παράλληλη εκτέλεση στόχων:** Στο πολυπρακτορικό, οι πράκτορες μπορούν να εκτελούν διαφορετικούς στόχους ταυτόχρονα (π.χ. συλλογή αντικειμένων για διαφορετικά tasks), μειώνοντας τον συνολικό χρόνο ολοκλήρωσης.
- **Overhead συντονισμού:** Το πολυπρακτορικό εισάγει κόστος επικοινωνίας (messages + wait states) και επανεκκινήσεις διαπραγμάτευσης, κάτι που δεν υπάρχει στο μονοπρακτορικό.
- **Συγκρούσεις και αποφυγή:** Στο μονοπρακτορικό δεν υπάρχει collision constraint. Στο πολυπρακτορικό, κάθε πράκτορας αντιμετωπίζει τον άλλον ως δυναμικό εμπόδιο, με πιθανές παρακάμψεις και extra βήματα.
- **Ανθεκτικότητα σε μετακινήσεις αντικειμένων:** Με 2 πράκτορες υπάρχει καλύτερη κάλυψη του χώρου (coverage) και συχνά πιο γρήγορη επανα-εύρεση αντικειμένων που αλλάζουν θέση, όμως αυξάνεται η πιθανότητα “να κυνηγάνε” το ίδιο αντικείμενο αν δεν γίνει σωστή ανάθεση.
- **Αποφυγή αδιεξόδων:** Το πρωτόκολλο περιλαμβάνει μηχανισμό ασφάλειας (όριο επαναλήψεων $N \geq 15$) ώστε να αποφεύγεται άπειρη διαπραγμάτευση. Στο μονοπρακτορικό αυτό δεν απαιτείται.
- **Συνολικό αποτέλεσμα:** Το πολυπρακτορικό αναμένεται να μειώνει τον συνολικό αριθμό βημάτων μέχρι την ολοκλήρωση όλων των στόχων, αλλά μπορεί να αυξάνει τον αριθμό ενεργειών/μηνυμάτων ανά επεισόδιο λόγω συντονισμού.

5 Προβλήματα κατά την Υλοποίηση και Αντιμετώπιση

Κατά τη διάρκεια των δοκιμών και της ανάπτυξης του συστήματος, αντιμετωπίστηκαν σημαντικές προκλήσεις που αφορούσαν τη δυναμική φύση του περιβάλλοντος και τους περιορισμούς του πλέγματος.

5.1 Επικάλυψη Αντικειμένων και Στατικών Στόχων

Ένα από τα κύρια προβλήματα ήταν η εμφάνιση κινητών αντικειμένων (όπως το brush ή το color) πάνω στα κελιά που καταλάμβαναν τα στατικά αντικείμενα (table, chair, door).

- **Πρόβλημα:** Όταν ένα αντικείμενο βρισκόταν «κάτω» από ένα τραπέζι, ο πράκτορας δυσκολευόταν να το εντοπίσει ή να το συλλέξει, καθώς το μοντέλο του περιβάλλοντος συχνά έδινε προτεραιότητα στην απεικόνιση του στατικού αντικειμένου.
- **Αντιμετώπιση:** Τροποποιήθηκε η λογική τοποθέτησης και η μέθοδος `cellOk` στην Java, ώστε να επιτρέπεται η συνύπαρξη αντικειμένων αλλά να διασφαλίζεται ότι ο πράκτορας μπορεί να εκτελέσει την ενέργεια `pick` ακόμα και σε «πολυσύχναστα» κελιά.

5.2 Δυναμική Μετακίνηση Αντικειμένων (Stuck Agents)

Λόγω της μεθόδου `stepDynamics`, τα αντικείμενα άλλαζαν θέση κάθε λίγα βήματα.

- **Πρόβλημα:** Πολλές φορές, τη στιγμή που ο πράκτορας έφτανε στο κελλί-στόχο, το αντικείμενο μετακινείτο σε γειτονικό κελλί. Αυτό οδηγούσε σε έναν φαύλο κύκλο «καταδίωξης» (chasing) όπου ο πράκτορας έμενε στάσιμος ή εγκλωβιζόταν σε επαναλαμβανόμενες κινήσεις (livelock).
- **Αντιμετώπιση:** Εισήχθησαν καθυστερήσεις (`.wait`) και επαναληπτικός σχεδιασμός στον κώδικα `AgentSpeak` ώστε ο πράκτορας να επανεκτιμά τη θέση του αντικειμένου αν η ενέργεια `pick` αποτύχει.

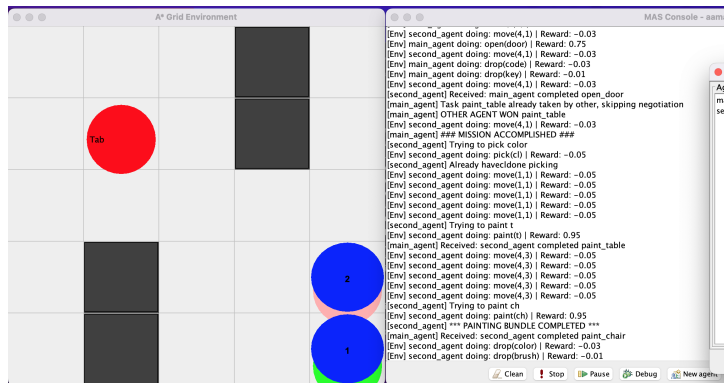
5.3 Περιορισμοί στην Προσέγγιση Στόχων

Παρατηρήθηκε ότι ο πράκτορας μερικές φορές αρνείτο να προσεγγίσει την πόρτα ή τα αντικείμενα βαφής, παραμένοντας σε συγκεκριμένα σημεία γύρω από αυτά.

- **Αιτία:** Η αυστηρή αποφυγή συγκρούσεων και η θεώρηση των αντικειμένων ως εμποδίων από τον αλγόριθμο A^* εμπόδιζαν τον πράκτορα να πατήσει ακριβώς πάνω στο κελλί του στόχου.
- **Λύση:** Ενημερώθηκε η λογική του περιβάλλοντος ώστε τα κελιά-στόχοι (`d`, `t`, `ch`) να θεωρούνται προσπελάσιμα (`passable`) μόνο όταν ο πράκτορας έχει τον αντίστοιχο στόχο εκτέλεσης.

6 Ανάλυση Επιτυχούς Ολοκλήρωσης

Παρά τις παραπάνω δυσκολίες, το τελικό σύστημα επέδειξε αξιοσημείωτη επιτυχία στην ολοκλήρωση του σεναρίου.



Σχήμα 3: Στιγμιότυπο επιτυχούς ολοκλήρωσης: Η πόρτα είναι ανοιχτή και τα αντικείμενα έχουν βαφτεί.

Σημεία Επιτυχίας:

- **Αποτελεσματική Διαπραγμάτευση:** Οι πράκτορες κατάφεραν να μοιράσουν τις εργασίες χωρίς επικαλύψεις. Ο `main_agent` συνήθως αναλάμβανε το άνοιγμα της πόρτας λόγω θέσης, ενώ ο `second_agent` επικεντρωνόταν στη βαφή.
- **Συντονισμός:** Η χρήση των `wait states` επέτρεψε στους πράκτορες να «συνομιλούν» χωρίς να κρασάρει το σύστημα, ακόμα και όταν οι χρόνοι απόκρισης του περιβάλλοντος διέφεραν.
- **Τελική Κατάσταση:** Σε όλα τα επιτυχή επεισόδια, οι πράκτορες αφού ολοκλήρωναν τα καθήκοντά τους, προχωρούσαν σε `drop_all` των αντικειμένων τους, αφήνοντας το περιβάλλον σε καθαρή τελική κατάσταση.

6.1 Διαχείριση Αποτυχιών και Μηχανισμοί Ανάνηψης (Failure Handling)

Ένα κρίσιμο σημείο της υλοποίησης ήταν η διαχείριση των αποτυχιών κατά την εκτέλεση των σχεδίων (plans). Στα συστήματα BDI, μια αποτυχία μπορεί να οδηγήσει σε πλήρη παύση του πράκτορα αν δεν υπάρξει πρόβλεψη.

- **Πρόβλημα «Απρόσμενης Αποτυχίας» (Unexpected Failure):** Στις δοκιμές παρατηρήθηκε το φαινόμενο `[main_agent] Unexpected failure`. Αυτό συνέβαινε όταν ο πράκτορας προσπαθούσε να εκτελέσει μια ενέργεια (π.χ. `paint` ή `pick`) αλλά οι προϋποθέσεις στο περιβάλλον είχαν αλλάξει ακαριαία (π.χ. το αντικείμενο μετακινήθηκε λόγω δυναμικής ή ο άλλος πράκτορας εμποδίζει την πρόσβαση).
- **Αποτυχία Κίνησης:** Λόγω του περιορισμένου χώρου (5×5), συχνά οι πράκτορες εγκλωβίζονταν σε σημεία όπου ο αλγόριθμος A* δεν μπορούσε να βρει μονοπάτι (No path found).
- **Μηχανισμός Ανάνηψης (Recovery):** Για την αντιμετώπιση αυτών, χρησιμοποιήθηκαν πλάνα διαχείρισης αποτυχιών (`¬!goal`) στον κώδικα `AgentSpeak`:

- Αρκετες φορές οι πράκτορες δεν μπορούσαν να μετακινηθούν διότι "μπλοκαρουντουσαν" όταν πηγαινανε πάνω σε ένα αντικείμενο κι αυτό το αντικείμενο μετακινούνται
- Αρκετες φορές οι πράκτορες δεν μπορούσαν να κάνουν τον στόχο διότι μπλοκαριζόταν από τον άλλο πράκτορα
- αν ένας πράκτορας μετακινούτνα μόνο τότε όταν ολοκλήρωνε την διαδικασία πέφταμε σε deadlock γιατι μπαινανε και οι δυο σε κατάσταση αναμονής για το ποιος θα παρει το επομενο τασκ

7 Συμπεράσματα

Η μετάβαση από το μονοπρακτορικό στο πολυπρακτορικό σύστημα ανέδειξε την ανάγκη για στιβαρά πρωτόκολλα επικοινωνίας. Παρόλο που το περιβάλλον έγινε πιο «θορυβώδες» λόγω των δυναμικών μετακινήσεων, η ικανότητα των πρακτόρων να διαπραγματεύονται και να προσαρμόζουν τα πλάνα τους οδήγησε σε ταχύτερη ολοκλήρωση των στόχων σε σύγκριση με την εκτέλεση από έναν μόνο πράκτορα.